示例 - T800-Odin2激光雷达相机

Odin ROS Driver

中文：基于 Odin SDK 的 ROS 驱动程序，支持 ROS1 和 ROS2 多版本。

English: ROS driver built on top of the Odin SDK, supporting both ROS1 and ROS2.

功能特性 / Features

- **多平台支持 / Multi-platform**: ROS1 Noetic、ROS2 Humble/Iron/Jazzy
- **热插拔检测 / Hot-plug**: 自动发现和连接设备，支持设备断开重连 / Auto-discovery and reconnection of devices
- **心跳监控 / Heartbeat**: 实时监测设备连接状态 / Real-time device connection monitoring
- **多数据流 / Multi-stream**: 点云、图像、IMU、里程计同步输出 / Synchronized point cloud, image, IMU and odometry output
- **后处理 / Post-processing**: 图像去畸变、点云着色、立体校正、点云转深度图 / Image undistortion, cloud colorization, stereo rectification, point-cloud → depth

功能模块 / Nodes

节点 / Node	说明 / Description
`odin_ros_driver_node`	主驱动节点：连接设备、发布原始数据、提供标定服务 / Main driver: device connection, raw data publishing, calibration service
`post_process_node`	后处理节点：图像去畸变、点云着色、立体校正 / Post-processing: undistortion, cloud colorization, stereo rectification
`pointcloud_reprojector_node`	点云重投影节点：稀疏点云重投影到 `camera0` 输出叠加可视化图 / Reprojects sparse point cloud onto `camera0` to produce an overlay visualization image
`pointcloud_to_depth_node`	点云转深度图节点（多设备）：投影 + 膨胀 + Sobel 边缘抑制 + 最近邻上采样 / Multi-device point-cloud → depth: projection + dilation + Sobel edge suppression + NN upsample

系统要求 / System Requirements

Ubuntu 版本 / Version	ROS 版本 / ROS Version
20.04	ROS1 Noetic, ROS2 Foxy
22.04	ROS2 Humble, ROS2 Iron
24.04	ROS2 Jazzy

连接主板 / Connect to the Orin Board

ODIN2 连接在 T800 系列的 Orin 主板上。ODIN2 出厂默认 IP 为 `192.168.100.251`，Orin 主板 IP 为 `192.168.0.162`。可通过 SSH 登录 Orin 主板进行调试和运行：

ODIN2 is connected to the Orin board in the T800 series. The factory default IP address of ODIN2 is `192.168.100.251`, and the Orin board IP address is `192.168.0.162`. Use SSH to log in to the Orin board for debugging and operation:

Code block

```
ssh -X username@192.168.0.162
```

ROS1

Code block

```
source /opt/ros/noetic/setup.bash
cd your_ros_workspace
./src/odin_ros_driver/script/build_ros1.sh
```

ROS2

Code block

```
source /opt/ros/humble/setup.bash
cd your_ros_workspace
./src/odin_ros_driver/script/build_ros2.sh
```

运行 / Run

ROS1

Code block

```
source devel/setup.bash
# 默认配置（自动发现设备，使用设备默认分辨率）
# Defaults (auto-discover device, use device-default resolution)
roslaunch odin_ros_driver_rev1 driver.launch

# 指定图像分辨率和帧率 / Specify image resolution and frame rate
roslaunch odin_ros_driver_rev1 driver.launch image_width:=640
image_height:=544 image_fps:=30

# 不启动 RViz / Do not launch RViz
roslaunch odin_ros_driver_rev1 driver.launch start_rviz:=false
```

ROS2

Code block

```
source install/setup.bash
# 默认配置（自动发现设备，使用设备默认分辨率）
# Defaults (auto-discover device, use device-default resolution)
ros2 launch odin_ros_driver_rev1 driver.launch.py

# 指定图像分辨率和帧率 / Specify image resolution and frame rate
ros2 launch odin_ros_driver_rev1 driver.launch.py image_width:=640
image_height:=544 image_fps:=30

# 不启动 RViz / Do not launch RViz
ros2 launch odin_ros_driver_rev1 driver.launch.py start_rviz:=false
```

Launch 参数说明 / Launch Parameters

中文：launch 文件已把全部可调参数暴露为 `<arg>` / `DeclareLaunchArgument`，可以通过 `key:=value` 在命令行覆盖。下表只列最常用的几个；**完整列表见下文 参数 / Parameters 章节**。

English: Every tunable knob is exposed as a launch argument and can be overridden via `key:=value` on the command line. The table below lists only the most common ones; **see Parameters below for the complete list**.

参数 / Parameter	默认值 / Default	说明 / Description
`start_rviz`	`true`	是否启动 RViz / Whether to start RViz

`topic_prefix`	`manifold`	话题前缀根 / Topic prefix root
`image_width`	`0`	图像宽度，`0` = 使用设备能力表第一项；具体取值需在线查询（见下方说明）/ Image width, `0` = first entry of the device's capability table; query online for actual values (see note below)
`image_height`	`0`	图像高度，`0` = 使用设备能力表第一项 / Image height, `0` = first capability entry
`image_fps`	`0`	图像帧率，`0` = 使用设备能力表第一项 / Image frame rate, `0` = first capability entry
`image_format`	`mjpeg`	图像格式（5 选 1，详见下方枚举）/ Image format (one of 5, see enum below): `mjpeg` | `yuyv` | `nv12` | `nv21` | `rgb24`
`operating_mode`	`normal`	工作模式（2 选 1）/ Operating mode (one of 2): `normal` | `standby`
`sensor_mode`	`0`	dToF 传感器模式（4 选 1，运行期可调）/ dToF sensor mode (one of 4, runtime-tunable): `0` =HDR | `1` =High Peak | `2` =Low Peak | `3` =Adaptive
`confidence_threshold`	`7`	点云置信度阈值（`0..255`，运行期可调）/ Point-cloud confidence threshold (`0..255`, runtime-tunable)
`log_level`	`1`	日志级别（4 选 1，运行期可调）/ Log level (one of 4, runtime-tunable): `0` =ERROR | `1` =INFO | `2` =WARN | `3` =DEBUG
`publish_tf_odom` / `publish_tf_map`	`true` / `true`	TF 发布开关 / TF publish switches: `true` | `false`

`image_*` 取值说明 / Notes on `image_*` values

设备实际支持的分辨率与帧率组合**因设备型号、固件版本而异**，README 不固化清单；驱动启动时会调用 SDK 接口 `sdk::GetSensorCapability(handle, caps, {channel}, timeout_ms)` 拉取能力表，并通过 `QueryAndPrintSensorCapabilities()` 打印到终端。请以**当次启动日志**为准选择 `image_width / image_height / image_fps`，否则会被设备拒绝、回落到默认能力。

The exact resolution/FPS combinations a device supports **vary by model and firmware**, so this README does not freeze the list. On startup the driver calls `sdk::GetSensorCapability(handle, caps, {channel}, timeout_ms)` and prints the capability table via `QueryAndPrintSensorCapabilities()`. Always pick `image_width / image_height / image_fps` from **the current run's log**; otherwise the device will reject the combination and fall back to its default mode.

典型 Odin2 能力（仅供参考，以实际查询为准）/ Typical Odin2 capabilities (reference only, see runtime query):

Channel	分辨率 / Resolution	帧率 / FPS	默认格式 / Default format
`Image0` (左 / left)	`1280×1088`, `640×544`	`30`	`mjpeg` (`OdinDataFormat::kImageMjpeg = 0x00`)
`Image1` (右 / right)	`1280×1088`, `640×544`	`30`	`mjpeg`

来源 / Source: `@/home/xxx/work/odin2/ros_fold/src/ros_driver/module/sdk_api/docs/protocol/PROTOCOL.md:198-213`

`image_format` 完整枚举 / Full `image_format` enumeration (`OdinDataFormat` in `@/home/xxx/work/odin2/ros_fold/src/ros_driver/module/sdk_api/include/odin_lidar_def.h:105-111`):

launch 字符串 / launch string	SDK 枚举 / SDK enum	编码值 / value	说明 / Note
`mjpeg` (默认 / default)	`kImageMjpeg`	`0x00`	MJPEG 压缩，带宽最低 / lowest bandwidth
`yuyv`	`kImageYuyv`	`0x01`	YUV 4:2:2 raw
`nv12`	`kImageNv12`	`0x02`	YUV 4:2:0, Y plane + interleaved UV

`nv21`	`kImageNv21`	`0x03`	YUV 4:2:0, Y plane + interleaved VU
`rgb24`	`kImageRgb24`	`0x04`	8-bit RGB

字符串解析见 / String parsing: `@/home/xxx/work/odin2/ros_fold/src/ros_driver/src/odin_ros_driver_node.cpp:933-943`；未识别字符串一律回落到 `mjpeg` / unknown strings fall back to `mjpeg`.

运行期获取设备能力的两种方法 / Two ways to inspect device capabilities at runtime:

1. **看驱动启动日志 / Read driver startup log** —— 搜索 `Sensor capabilities query complete` 或 `╔══...╗` ASCII 框；驱动每次启动都会自动打印当前设备的全部 channel 能力表。
   Search for `Sensor capabilities query complete` or the ASCII box header `╔══...╗` in the console. The driver prints every channel's capability table on every startup.
2. **在自己的代码里直接调用 SDK / Call the SDK directly:**

Code block

```
std::vector<sdk::OdinSensorCapability> caps;
sdk::GetSensorCapability(device_handle, caps,
    {sdk::OdinDataChannel::kImage0}, /*timeout_ms=*/1000);
for (const auto& m : caps[0].modes) {
  printf("%ux%u@%u fmt=%d\n", m.width, m.height, m.fps, (int)m.format);
}
```

> ⚠️ 不要把 README 里"典型 Odin2 能力"硬编码到客户脚本中 —— 不同设备/固件可能差异很大，唯一可信来源是 sdk::GetSensorCapability 的运行期返回。
>
> ⚠️ Do not hard-code the "Typical Odin2 capabilities" table into your scripts — values may differ across devices/firmware. The only authoritative source is the runtime return of sdk::GetSensorCapability.

发布话题 / Published Topics

中文：所有数据话题均带有逐设备前缀 `{prefix} = /{topic_prefix}/{model}/device{N}/`（默认 `/manifold/odin/device0/`），用于多设备热插拔区分。`topic_prefix` 由 launch 参数 `topic_prefix` 配置（默认 `manifold`），`{model}` 为设备型号小写（如 `odin`），`{N}` 为枚举到的设备序号。下表中的话题列均省略前缀，只列后缀。

English: All data topics carry a per-device prefix `{prefix} = /{topic_prefix}/{model}/device{N}/` (default `/manifold/odin/device0/`) so that

multiple devices can coexist. `topic_prefix` is configurable via the launch parameter `topic_prefix` (default `manifold`); `{model}` is the lowercased device model (e.g. `odin`); `{N}` is the enumerated device index. The tables below list only the suffix.

odin_ros_driver_node（主驱动节点 / main driver node）

话题后缀 / Topic suffix	类型 / Type	说明 / Description
`cloud/raw`	`sensor_msgs/PointCloud2`	原始点云（256×192）/ Raw point cloud (256×192)
`cloud/slam`	`sensor_msgs/PointCloud2`	SLAM 点云 / SLAM point cloud
`camera0/compressed`	`sensor_msgs/CompressedImage`	左相机 JPEG 压缩图像 / Left camera JPEG compressed image
`camera1/compressed`	`sensor_msgs/CompressedImage`	右相机 JPEG 压缩图像 / Right camera JPEG compressed image
`gray`	`sensor_msgs/Image`	灰度图像（mono8）/ Gray image (mono8)
`imu`	`sensor_msgs/Imu`	IMU 数据（加速度、角速度）/ IMU data (acceleration, angular velocity)
`odometry`	`nav_msgs/Odometry`	里程计数据 / Odometry data
`odometry_hf`	`nav_msgs/Odometry`	高频里程计 / High-frequency odometry
`odometry_lidar2cam`	`nav_msgs/Odometry`	相机端在线估算的雷达→相机外参（仅变换，无速度；静态出厂值见标定文件）/ Online-estimated LiDAR→camera extrinsic from the camera (transform-only; static factory value is in the calibration file)
`odometry_lidar2imu`	`nav_msgs/Odometry`	相机端在线估算的雷达→IMU 外参（仅变换，无速度；静态出厂值见标定文件）/ Online-estimated LiDAR→IMU extrinsic from the camera (transform-only; static factory value is in the calibration file)

此外，主驱动还会在驱动级命名空间 `/{topic_prefix}/driver/` 下发布以下话题（不带设备前缀，所有设备共享）：

The main driver also publishes the following topics under the driver-level namespace `/{topic_prefix}/driver/` (not per-device, shared across devices):

话题 / Topic	类型 / Type	说明 / Description
`/{topic_prefix}/driver/device_online`	`std_msgs/String`	设备上线事件（latched，JSON 含 SN/prefix/IP）/ Device-online event (latched, JSON with SN/prefix/IP)
`/{topic_prefix}/driver/device_offline`	`std_msgs/String`	设备下线事件（latched）/ Device-offline event (latched)
`/{topic_prefix}/driver/resolution_change`	`std_msgs/String`	分辨率变更事件（latched）/ Resolution change event (latched)

post_process_node（后处理节点 / post-processing node）

话题后缀 / Topic suffix	类型 / Type	说明 / Description
`camera0/raw`	`sensor_msgs/Image`	左相机解码后 RGB 图像 / Left camera decoded RGB image
`camera1/raw`	`sensor_msgs/Image`	右相机解码后 RGB 图像 / Right camera decoded RGB image
`camera0/undistort`	`sensor_msgs/Image`	左相机去畸变图像 / Left camera undistorted image
`camera1/undistort`	`sensor_msgs/Image`	右相机去畸变图像 / Right camera undistorted image
`camera0/rect`	`sensor_msgs/Image`	立体校正后左图像 / Stereo-rectified left image
`camera1/rect`	`sensor_msgs/Image`	立体校正后右图像 / Stereo-rectified right image
`camera0/camera_info`	`sensor_msgs/CameraInfo`	左相机校正后内参 / Rectified left camera intrinsics
`camera1/camera_info`	`sensor_msgs/CameraInfo`	

		右相机校正后内参 / Rectified right camera intrinsics
`cloud/render`	`sensor_msgs/PointCloud2`	着色点云（点云+图像融合）Lidar 坐标系 / Colored point cloud (cloud + image fusion) Lidar Frame
`stereo/depth`	`sensor_msgs/PointCloud2`	视差转点云输出 / Disparity → point cloud output

pointcloud_reprojector_node（点云重投影节点 / point-cloud reprojector）

中文：把稀疏 dToF 点云重投影到 `camera0` 图像平面，输出叠加可视化图，用于标定校验。支持两种模式： `raw` （ `cloud/raw` + 静态外参）和 `slam` （ `cloud/slam` + 实时 odom，世界坐标 → 相机坐标）。源码位于 `src/pointcloud_reprojector/` 。

English: Reprojects the sparse dToF point cloud onto `camera0` to produce a visualization overlay, useful for calibration verification. Two modes: `raw` (`cloud/raw` + static extrinsic) and `slam` (`cloud/slam` + live odometry, world → camera). Source under `src/pointcloud_reprojector/` .

话题后缀 / Topic suffix	类型 / Type	说明 / Description
`reprojection/overlay`	`sensor_msgs/Image` (bgr8)	点云叠加在相机图上 / cloud overlaid on the camera image

主要参数 / Key parameters: `mode` (`raw` / `slam`), `topic_prefix` , `min_depth` , `max_depth` , `point_size` , `cloud_topic_suffix` , `image_topic_suffix` , `odom_topic_suffix` 。

pointcloud_to_depth_node（点云转深度图节点 / point-cloud → depth）

中文：基于纯 C 风格的投影核心 (`pointcloud_to_depth_node.hpp`)，把同步好的 `cloud/raw` 与 `camera0/undistort` 投影到 `1/scale` 分辨率的低分画布上，再经膨胀、Sobel 边缘抑制、最近邻上采样得到与相机同分辨率的稠密深度图。**支持多设备**：每个 SN 一个 `DeviceWorker` ，通过 `device_online` / `device_offline` 动态生灭， `resolution_change` 按 SN 分发。源码位于 `src/pointcloud_to_depth/` ，详见同目录下 `README.md` 。

English: A library-free point-cloud → depth projection node. Synchronizes `cloud/raw` with `camera0/undistort` , projects onto a `1/scale` low-resolution canvas, then dilates / Sobel-edge-suppresses / nearest-neighbor-upsamples to a full-resolution dense depth map. **Multi-device aware**: one `DeviceWorker` per SN, spawned / torn down on `device_online` /

`device_offline`; `resolution_change` is dispatched by SN. The plain-types projection core (`pointcloud_to_depth_node.hpp`) is PCL/Eigen/ROS-free so it can be lifted into the on-device SDK. Source under `src/pointcloud_to_depth/`; see the in-folder `README.md` for the algorithm details.

话题后缀 / Topic suffix	类型 / Type	说明 / Description
`pointcloud2depth/depth`	`sensor_msgs/Image` (32FC1)	稠密深度图，米，0 表示无效像素 / dense depth (meters), 0 = invalid
`pointcloud2depth/depth_color`	`sensor_msgs/Image` (bgr8)	JET 配色可视化（无效像素为黑）/ JET-colored depth (black where invalid)
`pointcloud2depth/depth_overlay`	`sensor_msgs/Image` (bgr8)	JET 叠加在相机图上 / JET overlay on the RGB image

订阅 / Subscribes: `{prefix}cloud/raw`, `{prefix}camera0/undistort`, `/{topic_prefix}/driver/device_online`, `/{topic_prefix}/driver/device_offline`, `/{topic_prefix}/driver/resolution_change`。
依赖服务 / Service: `{prefix}get_calibration`（首次上线时拉取相机内外参 / fetched once per device on first online event）。

主要参数 / Key parameters:

参数 / Parameter	默认值 / Default	说明 / Description
`scale`	`7.0`	投影画布下采样比 / projection canvas downscale factor
`z_min` / `z_max`	`0.1` / `50.0`	有效深度范围（米）/ valid depth range (meters)
`dilate_radius`	`1`	投影写入时的 3×3 膨胀 / dilation radius when writing projected pixels
`use_z_buffer`	`false`	是否启用 Z-buffer 防止远点覆盖近点 / enable z-buffer to prevent far-over-near overwrite
`edge_grad_threshold`	`0.75`	Sobel 边缘抑制阈值（米/像素）/ Sobel edge suppression

		threshold (m/pixel)
`vis_depth_min` / `vis_depth_max`	`0.1` / `5.0`	JET 配色固定归一化范围；`<0` 时改为逐帧自动归一化 / fixed JET normalization range; set min `<0` for per-frame auto-norm
`sync_queue_size` / `sync_slop_sec`	`10` / `0.1`	message_filters 时间同步参数 / time-sync queue size & slop

服务 / Services

所有服务均带有逐设备前缀 `{prefix} = /{topic_prefix}/{model}/device{N}/`（与话题相同）。下文示例使用默认前缀 `/manifold/odin/device0/`。

All services carry the same per-device prefix `{prefix} = /{topic_prefix}/{model}/device{N}/` as the topics. The examples below use the default prefix `/manifold/odin/device0/`.

服务后缀 / Service suffix	类型 / Type	说明 / Description
`get_calibration`	`odin_ros_driver_rev1/srv/GetCalibration`	获取相机标定 YAML 内容 / Fetch the camera calibration YAML content
`set_sensor_mode`	`odin_ros_driver_rev1/srv/SetSensorMode`	设置 dToF 传感器工作模式 / Set the dToF sensor operating mode

`get_calibration`

请求空、响应内容（Request empty; Response）：

Code block

```
# Request: (empty)
---
bool   success
string yaml_content   # 标定文件原始 YAML 内容 / raw YAML calibration content
string message
```

ROS2 调用示例 / ROS2 call:

Code block

```
ros2 service call /manifold/odin/device0/get_calibration \
```

```
  odin_ros_driver_rev1/srv/GetCalibration "{}"
```

ROS1 调用示例 / ROS1 call:

Code block

```
rosservice call /manifold/odin/device0/get_calibration "{}"
```

`set_sensor_mode`

Code block

```
uint8 mode   # 0=HDR 1=High Peak  2=Low Peak  3=Adaptive
---
bool   success
int32  error_code   # 0=OK, 其它为 SDK OdinResult 枚举 / non-zero from SDK OdinResult enum
string message
```

ROS2 调用示例 / ROS2 call:

Code block

```
# 切换到 Adaptive 模式 / Switch to Adaptive mode
ros2 service call /manifold/odin/device0/set_sensor_mode \
  odin_ros_driver_rev1/srv/SetSensorMode "{mode: 3}"
```

ROS1 调用示例 / ROS1 call:

Code block

```
rosservice call /manifold/odin/device0/set_sensor_mode "mode: 3"
```

> 提示 / Note：set_sensor_mode 与下文的 sensor_mode 参数功能等价；二者任一被修改时，驱动都会向设备下发 sdk::SetSensorMode。
>
> The set_sensor_mode service is equivalent to the sensor_mode parameter below; modifying either of them triggers sdk::SetSensorMode on the device.

参数 / Parameters

参数分两类：**启动期参数（launch-time）**只在节点启动时读取一次；**运行期参数（runtime / dynamic）**可以在节点运行中通过 `ros2 param set`（ROS2）或 `rqt_reconfigure`（ROS1 ddynamic_reconfigure）随时改写。

> ⚡ 支持动态回调的参数（whitelist） / Parameters with a dynamic callback
>
> 当前只有以下 3 个参数注册了动态回调，运行期 ros2 param set / rqt_reconfigure 改完会立即下发到 SDK / 日志系统：
>
> Currently only the 3 parameters below have a dynamic callback. Changing them at runtime via ros2 param set / rqt_reconfigure takes effect immediately:

参数 / Parameter	节点 / Node	作用 / Effect
`confidence_threshold`	`odin_ros_driver_node`	`sdk::SetConfidenceThreshold`
`sensor_mode`	`odin_ros_driver_node`	`sdk::SetSensorMode`
`log_level`	`odin_ros_driver_node`	切换 SDK / ROS 日志级别

> 其余所有参数都是启动期参数——ros2 param set 会成功写入参数服务器，但不会回流到节点逻辑。如果要改它们，请通过 launch/driver.launch.py（ROS2）或 launch/driver.launch（ROS1）的 key:=value 透传，或修改 launch 文件的默认值后重启节点。
>
> All other parameters are launch-time only — ros2 param set will update the parameter server but will not flow into the node logic. To change them, override via launch/driver.launch.py (ROS2) or launch/driver.launch (ROS1) using key:=value, or edit the launch defaults and restart.

下表中**所有参数都已在 `launch/driver.launch.py`（ROS2）和 `launch/driver.launch`（ROS1）中暴露为 launch arg**，可通过 `ros2 launch ... key:=value` / `roslaunch ... key:=value` 在命令行直接覆盖默认值。

> 命名约定 / Naming convention：launch arg 名与节点参数名始终保持一致（不加前缀）。绝大多数参数只属于某一个节点，无歧义。唯一跨节点同名的参数是 sync_queue_size（pointcloud_reprojector_node + pointcloud_to_depth_node），在 launch 中只声明一次，同时下发给两个节点；如需分别设置，用节点全路径覆盖：ros2 param set /pointcloud_to_depth sync_queue_size 20。
>
> Launch arg names are kept identical to node parameter names (no prefixes). Almost all parameters belong to a single node and are unambiguous. The only cross-node duplicate is sync_queue_size (pointcloud_reprojector_node + pointcloud_to_depth_node); it is declared once in the launch and fans out to both nodes. To set them independently, use the full node path: ros2 param set /pointcloud_to_depth sync_queue_size 20.

Parameters fall into two groups: **launch-time** parameters are read once at node startup; **runtime / dynamic** parameters can be changed while the node is running via `ros2 param set` (ROS2) or `rqt_reconfigure` (ROS1). All parameters listed below are exposed as launch arguments in both `launch/driver.launch.py` and `launch/driver.launch`.

主驱动节点 odin_ros_driver_node — 启动期参数 / Launch-time parameters

参数 / Parameter	类型 / Type	默认值 / Default	说明 / Description
`topic_prefix`	string	`manifold`	话题前缀根（最终前缀 `/{topic_prefix}/{model}/device{N}/`，留空去掉根段）/ Topic prefix root
`operating_mode`	string	`normal`	工作模式 / Operating mode: `normal` | `standby`
`discovery_timeout_ms`	int	`2000`	SDK 设备发现超时（ms）/ SDK device-discovery timeout
`image_width`	int	`0`	图像宽度，`0`=设备默认 / Image width, 0 = device default
`image_height`	int	`0`	图像高度，`0`=设备默认 / Image height, 0 = device default
`image_fps`	int	`0`	图像帧率，`0`=设备默认 / Image FPS, 0 = device default
`image_format`	string	`mjpeg`	图像格式 / Image format: `mjpeg` | `yuyv` | `nv12` | `nv21` | `rgb24`
`raw_frame_id`	string	`odom`	原始点云 frame_id / Frame id for raw cloud
`slam_frame_id`	string	`odom`	SLAM 点云 frame_id / Frame id for SLAM cloud

`image_frame_id`	string	`odom`	左相机图像 frame_id / Frame id for left image
`image2_frame_id`	string	`odom`	右相机图像 frame_id / Frame id for right image
`gray_image_frame_id`	string	`odom`	灰度图像 frame_id / Frame id for gray image
`imu_frame_id`	string	`odom`	IMU frame_id
`odom_frame_id`	string	`odom`	里程计 frame_id（odom→base_link 的父帧）/ Odometry parent frame
`odom_child_frame_id`	string	`base_link`	里程计 child frame_id
`map_frame_id`	string	`map`	TF map 帧名 / Map frame name
`publish_tf_odom`	bool	`true`	是否发布 `odom→base_link` TF / Publish odom→base_link TF
`publish_tf_map`	bool	`true`	是否发布 `map→odom` TF / Publish map→odom TF
`odom_position_scale`	double	`1e-6`	里程计位置缩放（SDK 原始单位 → 米）/ Position scale (SDK → meters)
`odom_orientation_scale`	double	`1e-6`	里程计姿态缩放 / Orientation scale
`odom_linear_velocity_scale`	double	`1e-6`	线速度缩放 / Linear-velocity scale
`odom_angular_velocity_scale`	double	`1e-6`	角速度缩放 / Angular-velocity scale

启动期参数命令行设置示例 / Setting launch-time parameters from CLI:

Code block

```
# ROS2: 通过 launch 透传 / via launch
ros2 launch odin_ros_driver_rev1 driver.launch.py \
  topic_prefix:=mainifold image_width:=640 image_height:=544 image_fps:=30 \
  raw_frame_id:=lidar imu_frame_id:=imu_link publish_tf_odom:=false

# ROS2: 列出全部 launch arg / list every launch arg
ros2 launch odin_ros_driver_rev1 driver.launch.py --show-args

# ROS2: 直接 ros2 run（不通过 launch）/ direct ros2 run
ros2 run odin_ros_driver_rev1 odin_ros_driver_node \
  --ros-args -p topic_prefix:=mainifold -p image_fps:=30

# ROS1: 通过 launch 透传 / via roslaunch
roslaunch odin_ros_driver_rev1 driver.launch \
  topic_prefix:=mainifold image_width:=640 image_height:=544 image_fps:=30 \
  raw_frame_id:=lidar imu_frame_id:=imu_link publish_tf_odom:=false
```

主驱动节点 odin_ros_driver_node — 运行期参数 / Runtime parameters

下列三个参数注册了**动态回调**，运行期间修改会立即下发到 SDK 或日志系统。

The three parameters below register a **dynamic callback** and take effect immediately when changed at runtime.

参数 / Parameter	类型 / Type	范围 / Range	默认值 / Default	说明 / Description
`confidence_threshold`	int	`0..255`	`7`	点云置信度阈值（低于该值的点会被丢弃）/ Point-cloud confidence threshold (points below are dropped)
`sensor_mode`	int	`0..3`	`0`	dToF 传感器模式 / dToF sensor mode: `0` =HDR `1` =High Peak `2` =Low Peak `3` =Adaptive

`log_level`	int	`0..3`	—	日志级别 / Log level: `0` =ERROR `1` =INFO `2` =WARN `3` =DEBUG

ROS2 运行期改参示例 / ROS2 runtime change:

Code block

```
# 列出所有参数 / List all parameters
ros2 param list /odin_ros_driver_node

# 读取 / Get
ros2 param get /odin_ros_driver_node sensor_mode

# 设置 / Set （node 名字以实际启动名字为准）
ros2 param set /odin_ros_driver_node sensor_mode 3
ros2 param set /odin_ros_driver_node confidence_threshold 15
ros2 param set /odin_ros_driver_node log_level 1
```

ROS1 运行期改参示例 / ROS1 runtime change:

Code block

```
# ROS1 通过 ddynamic_reconfigure 暴露，可用 rqt_reconfigure 或命令行:
# Exposed through ddynamic_reconfigure; use rqt_reconfigure or the CLI:
rosrun rqt_reconfigure rqt_reconfigure
# 或 / or
rosrun dynamic_reconfigure dynparam set /odin_ros_driver_node sensor_mode 3
rosrun dynamic_reconfigure dynparam set /odin_ros_driver_node
confidence_threshold 15
```

后处理 / 重投影 / 点云转深度图 节点参数

下表中的参数名即 launch arg 名，也即节点参数名（完全一致，不加前缀）。

In the tables below the parameter name is also the launch-arg name and the node parameter name (no prefixes).

`pointcloud_reprojector_node` 参数 / parameters

参数 / Parameter	类型 / Type	默认值 / Default	说明 / Description

`topic_prefix`	string	`manifold`	与主驱动一致 / Same root as the main driver
`mode`	string	`raw`	重投影模式 / Mode: `raw` | `slam`（同时被 `config/control_command.yaml` 中的 `reprojection_mode` 覆盖 / overridden by `reprojection_mode` in `control_command.yaml`）
`min_depth`	double	`0.1`	可视化最小深度（米）/ Visualization min depth (m)
`max_depth`	double	`50.0`	可视化最大深度（米）/ Visualization max depth (m)
`point_size`	int	`2`	叠加点像素半径 / Overlay point pixel radius
`use_depth_color`	bool	`true`	是否按深度上色（false=单色）/ Color by depth or use single color
`sync_queue_size` ⚠	int	`10`	时间同步队列长度（与 `pointcloud_to_depth_node` 共用一个 launch arg）/ Time-sync queue size (shares one launch arg with `pointcloud_to_depth_node`)
`sync_slop`	double	`0.1`	时间同步最大偏差（s）/ Max time-sync slop (s)

`pointcloud_to_depth_node` 参数 / parameters

参数取值与说明见上文 pointcloud_to_depth_node 章节。可调参数：`scale`、`z_min`、`z_max`、`dilate_radius`、`use_z_buffer`、`edge_grad_threshold`、`vis_depth_min`、`vis_depth_max`、`sync_queue_size` ⚠、`sync_slop_sec`。

Full defaults / descriptions are in the pointcloud_to_depth_node section above. Tunables: `scale`, `z_min`, `z_max`, `dilate_radius`, `use_z_buffer`, `edge_grad_threshold`, `vis_depth_min`, `vis_depth_max`, `sync_queue_size` ⚠, `sync_slop_sec`.

> ⚠ sync_queue_size 为 pointcloud_reprojector_node 与 pointcloud_to_depth_node 共用的 launch arg；从 launch 设置时会同时生效于两个节点。需分别设置请走全路径：ros2 param set /pointcloud_to_depth sync_queue_size 20。
>
> ⚠ sync_queue_size is shared between pointcloud_reprojector_node and pointcloud_to_depth_node; setting it via the launch arg applies to both nodes. To set them independently, use the full path: ros2 param set /pointcloud_to_depth sync_queue_size 20.

`depth_to_pointcloud_node` 参数 / parameters

参数 / Parameter	类型 / Type	默认值 / Default	说明 / Description
`step`	int	`4`	像素采样步长（越小越稠密）/ Pixel sampling stride (smaller = denser)
`d_min`	double	`0.1`	有效深度下限（米）/ Valid depth lower bound (m)
`d_max`	double	`100.0`	有效深度上限（米）/ Valid depth upper bound (m)

命令行示例 / CLI examples

Code block

```
# 通过 launch 一次性覆盖（推荐）/ Override via launch (recommended)
ros2 launch odin_ros_driver_rev1 driver.launch.py \
  mode:=slam max_depth:=20.0 \
  scale:=5.0 use_z_buffer:=true \
  step:=3

# 单独 ros2 run 子节点 / Run a single sub-node directly
ros2 run odin_ros_driver_rev1 pointcloud_to_depth_node \
  --ros-args -p scale:=5.0 -p z_max:=20.0 -p use_z_buffer:=true

# 运行时查询 / Inspect at runtime
ros2 param list /pointcloud_reprojector
ros2 param get  /pointcloud_to_depth scale

```

```
# 共享参数分别设置（全路径）/ Disambiguate a shared param via full path
ros2 param set /pointcloud_reprojector sync_queue_size 30
ros2 param set /pointcloud_to_depth   sync_queue_size 15
```

> 注意 / Note：pointcloud_reprojector_node、pointcloud_to_depth_node、depth_to_pointcloud_node 三个子节点的参数均只在节点启动时读取一次，没有动态回调。运行期 ros2 param set 会修改参数服务器中的值，但不会重新进入计算逻辑，需要重启节点才会生效。
>
> All three sub-nodes (pointcloud_reprojector_node, pointcloud_to_depth_node, depth_to_pointcloud_node) read their parameters only once at startup; there is no dynamic callback. A runtime ros2 param set will update the parameter server but will not flow into the node logic — restart the node to apply changes.

数据流开关（YAML，非 ROS 参数）/ Stream toggles (YAML, not ROS parameters)

启用/禁用各数据流和后处理子模块通过 `config/control_command.yaml` 中的 `register_keys` 配置（详见下文 Stream Configuration）。这些开关在节点启动时一次性读取，**不**通过 ROS 参数系统暴露。

Per-stream and per-post-process toggles live in `register_keys` of `config/control_command.yaml` (see Stream Configuration below). They are read once at startup and are **not** exposed via the ROS parameter system.

> 中文：设备发现、序列号、网卡选择、UDP 端口等都由 SDK 内部自动处理，无需也无法在 launch 中配置。节点会枚举所有发现到的设备并为每台单独建立 topic 前缀（多设备热插拔）。
>
> English: Device discovery, serial number, NIC selection and UDP ports are all handled internally by the SDK — they are neither required nor configurable from the launch file. The node enumerates every discovered device and assigns a per-device topic prefix (multi-device hot-plug).

配置文件 / Configuration Files

文件 / File	说明 / Description
`config/camera_calib.yaml`	相机标定文件（从设备自动获取）/ Camera calibration file (auto-fetched from device)
`launch/driver.launch`	ROS1 启动配置 / ROS1 launch configuration
`launch/driver.launch.py`	ROS2 启动配置 / ROS2 launch configuration
`rviz/odin.rviz`	RViz 可视化配置 / RViz visualization configuration

启动说明 / Startup Notes

1. `device_ip` 留空时自动广播发现设备 / When `device_ip` is empty, devices are auto-discovered via broadcast.
2. 连接后自动设置为 `normal` 模式开始数据推送 / Upon connection the device is set to `normal` mode and starts streaming data.
3. 默认启动 RViz，可通过 `start_rviz:=false` 禁用 / RViz starts by default; disable with `start_rviz:=false`.
4. Ctrl+C 退出时自动将设备设为 `standby` 模式 / On Ctrl+C exit the device is set back to `standby` mode.

目录结构 / Directory Layout

Code block

```
ros_driver/
├── config/                          # 配置文件 / Configuration files
│   └── control_command.yaml         # 数据流开关配置 / Per-stream enable flags
├── include/                         # 公共头文件 / Common headers
│   ├── camera/                      # 内参类型 / Intrinsic types
│   ├── service_types.h
│   └── utility/                     # 工具类（ros_compat, yaml loader 等）/ Utilities
├── launch/                          # 启动文件 / Launch files
├── module/sdk_api/                  # Odin SDK（git submodule，详见上文 SDK 获取章节）/ Odin SDK (git submodule, see "SDK Acquisition" above)
├── rviz/                            # RViz 配置 / RViz configs
├── script/                          # 编译脚本 / Build scripts
├── src/                             # 源代码 / Source code
│   ├── odin_ros_driver_node.cpp     # 主驱动节点（ROS1/ROS2 统一）/ Main driver (ROS1/ROS2 unified)
│   ├── post_process_node.cpp        # 后处理节点 / Post-processing node
│   ├── camera_model/                # 相机投影模型 / Camera projection models
│   ├── cloud_render/                # 点云着色 / Cloud colorization
│   ├── color_undistort/             # 图像去畸变 / Image undistortion
│   ├── decoder/                     # JPEG 解码 / JPEG decoding
│   ├── disparity_to_pointcloud/     # 视差→点云 / Disparity → point cloud
│   ├── pointcloud_reprojector/      # 点云重投影节点 / Point-cloud reprojector node
│   ├── pointcloud_to_depth/         # 点云转深度图节点（多设备）/ Point-cloud → depth (multi-device)
│   └── stereo_rectifier/            # 立体校正 / Stereo rectification
└── srv/                             # 自定义服务 / Custom services
```

数据流配置 / Stream Configuration

中文：通过 `config/control_command.yaml` 可配置启用/禁用各数据流。当前系统中的 `config` 目录位于 `/home/ubuntu/odin/src/ros_driver/` 下。

English: Enable/disable individual streams via `config/control_command.yaml`. On the current system, the `config` directory is located under `/home/ubuntu/odin/src/ros_driver/`.

Code block

```yaml
register_keys:
  raw_point: true       # 原始点云 / Raw point cloud
  slam_point: true      # SLAM 点云 / SLAM point cloud
  image0: true          # 左相机图像 / Left camera image
  image1: true          # 右相机图像 / Right camera image
  imu: true             # IMU 数据 / IMU data
  odom: true            # 里程计数据 / Odometry data
```

Q&A

1. 多台电脑话题数据干扰 / Topic collisions across multiple machines

问题 / Problem：多台电脑通过 WiFi/局域网连接，启动 ROS Driver 时话题相同导致数据干扰。

When multiple machines run the driver on the same WiFi/LAN, identical topic names cause cross-machine interference.

ROS2 解决方法 / ROS2 solution：

Code block

```bash
# 查看当前 DOMAIN_ID（空白或 0 为共享模式）
# Check current DOMAIN_ID (empty or 0 means shared)
echo $ROS_DOMAIN_ID

# 设置独立的 DOMAIN_ID（0-100，0 为共享）
# Set an isolated DOMAIN_ID (0-100; 0 = shared)
export ROS_DOMAIN_ID=1
```

ROS1 解决方法 / ROS1 solution：

Code block

```bash
# 方法一（推荐）/ Option 1 (recommended)
export ROS_LOCALHOST_ONLY=1

```

```
# 方法二：使用不同端口 / Option 2: use a different master port
roscore -p 11312
export ROS_MASTER_URI=http://localhost:11312
```

> 以上命令只对当前终端生效。永久设置请添加到 ~/.bashrc 或 /etc/profile。
> The commands above only affect the current shell. Append them to ~/.bashrc or /etc/profile to make them permanent.

2. 设备连接失败 / Device connection failure

问题 / Problem：无法发现或连接设备。 / Cannot discover or connect to the device.

解决方法 / Solution：

1. 确保设备与电脑在同一网段 / Make sure the device and host are on the same subnet.
2. 检查防火墙是否阻止 UDP 广播（端口 60000-60010）/ Check whether the firewall blocks UDP broadcast (ports 60000-60010).
3. 尝试指定设备 IP / Try specifying the device IP explicitly: `roslaunch odin_ros_driver_rev1 driver.launch device_ip:=192.168.1.251`

3. 图像分辨率不生效（ROS1）/ Image resolution not applied (ROS1)

问题 / Problem：ROS1 下设置 `image_width` / `image_height` 参数无效。 / The `image_width` / `image_height` parameters have no effect on ROS1.

解决方法 / Solution：升级到 v0.9.0 或更高版本，已修复此问题。 / Upgrade to v0.9.0 or later — the issue has been fixed.

4. rosbag 录制丢包 / rosbag Recording Packet Loss

问题 / Problem：使用 `rosbag record` 或 `ros2 bag record` 录制数据时，高频话题（如 IMU 400Hz）出现丢包或时间戳间隔异常。

When using `rosbag record` or `ros2 bag record`, high-frequency topics (e.g., IMU at 400Hz) experience packet loss or abnormal timestamp intervals.

ROS2 解决方法 / ROS2 Solution

ROS2 默认使用 DDS 协议，QoS 策略为 `BEST_EFFORT`，高频数据容易丢包。需要配置 QoS 覆盖文件：

ROS2 uses DDS protocol with default `BEST_EFFORT` QoS, which can cause packet loss for high-frequency data. You need to configure a QoS override file:

Code block

```
# 使用提供的 QoS 配置文件录制（配置文件位于 config/rosbag2_qos.yaml）
```

```
# Use the provided QoS configuration file for recording (located at config/rosbag2_qos.yaml)
ros2 bag record -a --qos-profile-overrides-path <path_to_ros_driver>/config/rosbag2_qos.yaml

# 示例 / Example:
ros2 bag record -a --qos-profile-overrides-path ~/catkin_ws/src/odin_ros_driver/config/rosbag2_qos.yaml
```

QoS 配置说明 / QoS Configuration：

- `reliability: reliable` - 确保数据可靠传输 / Ensures reliable data transmission
- `depth: 4000` - 队列深度，建议为频率的 10 倍 / Queue depth, recommended 10x the frequency

ROS1 解决方法 / ROS1 Solution

ROS1 使用 TCP 协议（TCPROS），天然保证数据可靠传输，**无需额外配置**。

ROS1 uses TCP protocol (TCPROS), which inherently guarantees reliable data transmission. **No additional configuration is needed.**

Code block

```
# ROS1 直接录制即可
# ROS1 can record directly
rosbag record -a

# 如果磁盘 IO 较慢，可增加缓冲区
# If disk IO is slow, increase buffer size
rosbag record -a --buffsize=256
```

为什么 ROS1 和 ROS2 操作不同？ / Why are ROS1 and ROS2 different?

特性 / Feature	ROS1	ROS2
传输协议 / Protocol	TCP (TCPROS)	DDS (UDP-based)
默认可靠性 / Default Reliability	可靠 / Reliable	尽力而为 / Best Effort
丢包可能性 / Packet Loss	低 / Low	高频数据可能丢包 / Possible for high-freq data
配置方式 / Configuration	无需配置 / No config needed	需要 QoS YAML 文件 / Requires QoS YAML

原理说明 / Technical Explanation：

- **ROS1**：基于 TCP，协议层保证数据按序到达，不会丢包
- **ROS2**：基于 DDS/UDP，默认 `BEST_EFFORT` 策略不保证送达，需要显式配置 `RELIABLE` 策略
- **ROS1**: Based on TCP, the protocol layer guarantees in-order delivery without packet loss
- **ROS2**: Based on DDS/UDP, default `BEST_EFFORT` policy does not guarantee delivery, explicit `RELIABLE` configuration is required

版本历史 / Version History

详见 CHANGELOG.md / See CHANGELOG.md for details.